# ML Model Deployment Strategies and Model Serving using FastAPI

### Titanic Survival Prediction — Assignment Report

Dataset: Titanic – Machine Learning from Disaster (Kaggle), using the provided train.csv.

## 1. Objective

Train a classification pipeline, save it as model.pkl, demonstrate online, batch and streaming inference, and expose predictions through a FastAPI REST API.

## 2. Dataset and preprocessing

The supplied dataset contains 891 labelled passenger records. The target is Survived (0 = did not survive, 1 = survived). The model uses Pclass, Sex, Age, SibSp, Parch, Fare and Embarked. Missing numeric values are imputed with the median, missing categorical values with the most frequent value, numeric fields are scaled, and categorical fields are one-hot encoded.

## 3. Model training and evaluation

A Random Forest classifier was trained with a stratified 80/20 train/test split (random_state=42). Training rows: 712; test rows: 179. Test accuracy: 0.8156 (81.56%). The preprocessing steps and classifier are saved together in model.pkl so the same pipeline is reused for all inference modes.

## 4. Inference strategies

**Online inference:** one held-out passenger is passed to the saved model and a prediction is returned immediately. Example prediction: 0 (Did Not Survive).

**Batch inference:** 20 held-out passenger records are predicted in one call and saved to batch_predictions.csv.

**Streaming inference:** 10 held-out records are processed one at a time, with a one-second pause between records, and each prediction is printed.

## 5. FastAPI model serving

GET /health returns the API health status. POST /predict validates passenger fields using Pydantic and returns a numeric prediction and readable label. Swagger UI is available at /docs after the server is started. client.py sends a sample request to /predict.

## 6. Observations

**Online vs Batch:** Online inference predicts one request at a time and is suitable for interactive, low-latency responses. Batch inference processes a group of records together and is efficient for bulk jobs.

**Streaming vs Batch:** Streaming inference handles incoming records individually as they arrive, typically continuously. Batch inference waits for a collection of records and processes them together.

**Why FastAPI?:** FastAPI provides a lightweight Python web framework, automatic OpenAPI/Swagger documentation, request validation with Pydantic, and convenient REST endpoints.

**Why reuse the same model?:** Reusing the saved pipeline keeps preprocessing and model behavior consistent, avoids retraining for each request, reduces latency, and makes results reproducible.

## 7. How to run

1. Install Python 3.10 or newer. 2. Run: pip install -r requirements.txt. 3. Run inference scripts from the project folder. 4. Start the API using: uvicorn app:app --reload. 5. Open http://127.0.0.1:8000/docs to test /health and

/predict. 6. In another terminal, run: python client.py.

# 8. Screenshots

The screenshots supplied with the assignment are included in the screenshots/ folder as evidence of the original runs. Review them before submission and add any additional screenshots your faculty requires.